

Homework 5

*Practice Final Exam**August 30, 2026*

Instructions

- **Format of the real exam.** You will be given three problems: one on sorting, one on hashing, and one on graphs. You will answer **exactly one** of them. You have **65 minutes**.
- **Permitted materials (allow-list).** During the exam you may have open *only*: (1) the exam PDF; (2) your own digital notes, submitted to Gradescope before the exam window opened; (3) the lecture PDFs linked on the course site; (4) Wolfram for hash modulo; (5) Excalidraw tab; (6) a Google/Word Doc for your answer sheet. Anything not on this list is prohibited, including textbooks, solution manuals, and any file downloaded during the exam.
- **Your own digital notes.** One PDF, at most six pages, written by you. Upload it to Gradescope by 11:59pm PT Thursday. You may use only the copy you uploaded. Notes not submitted in advance are subject to a 30pt penalty on the exam.
- **Environment.** The exam is individual. You may not consult anyone else during the exam. Share your entire desktop, not a single window; and disconnect additional monitors. No web browser except the exam PDF, code editor, calculator, Excalidraw, and your answer sheet (if on web). No internet search and no generative AI of any kind is permitted. You must export your answer sheet to a PDF and submit within your allotted slot.
- **Drawing.** All sketching (slot tables, traces, graphs) is done in Excalidraw and stays on screen. Does *not* need to be submitted.
- **Show the work, not just the answer.** A table with no probe sequence, a runtime with no derivation, or an invariant with no maintenance step earns no credit. This is scored the way a technical interview is scored: the reasoning *is* the answer. (This means you can be partially wrong to an extent, and that's okay, as long as your supporting justification is sound about where you went wrong).
- **Arithmetic** is exact integer arithmetic throughout. Do not round.
- You may cite any result proved in lecture by name (Dijkstra's correctness, the $\Omega(n \log n)$ comparison-sort lower bound, the master theorem, and so on) without reproving it, unless a part explicitly asks you to prove it.

Study Material and Scope

Anything on this list is fair game. If something not lectured appears, the problem statement will include a spec of the procedur. Nothing outside this list will be tested.

Sorting and searching

- Loop invariants: initialization, maintenance, termination
- Time-per-execution tables and Θ derivation; best / worst / average / expected case analysis
- Recurrences: unrolling, recursion trees, master theorem
- Selection, insertion, merge sort; quicksort and quickselect (partitioning, pivot choice, recurrences)
- Counting sort, radix sort in base b , Timsort, hybrid blocksort
- Binary search: invariant, first / last occurrence, counting occurrences, search on an answer space
- The value of sorted data: sorting as preprocessing, two-pointer sweeps over a sorted array
- Brute force as a baseline: exponential- and factorial-time search, and what a better algorithm has to beat
- Stability, in-place / auxiliary space, the $\Omega(n \log n)$ comparison lower bound, and adaptivity
- Any other well-known comparison or distribution sorts (with accompanying code).

Hashing

- Load factor α , expected probe counts, chaining vs. open addressing
- Linear probing, quadratic probing, double hashing; probe sequence as a permutation and

the role of primality

- Tombstones and deletion correctness
- Resizing and rehashing, including writing the rebuild in code; amortized $O(1)$ via the geometric sum (*not including invoking/proving a sum or series, though*)
- Cuckoo hashing: eviction chains, cycles, the $\alpha < \frac{1}{2}$ hypothesis
- Any other collision-resolution or table-growth policy

Graphs

- Definitions and terminology: directed vs. undirected, degree and out-degree, walks, paths, cycles, connectedness
- Adjacency matrix vs. adjacency list (and dictionary-of-sets): space, degree / neighbor cost, selection criteria
- BFS: layers, unweighted shortest paths, parent / predecessor arrays, the BFS tree, correctness
- DFS: discovery and finish times, edge classification, cycle detection
- Topological sort
- Connectivity, connected components
- Dijkstra: relaxation, heap implementation, $\Theta((|V| + |E|) \log |V|)$, where non-negativity is used
- A^* : admissibility and consistency
- Prim, Kruskal, the cut property, union-find; MST-based single-linkage clustering
- Reverse graph and graph transformations (product / layered constructions)

Category	Problem	Page
Sorting	1. The Scrambled Queue, From the Back	3
Hashing	2. Double Hashing With a Rebuild Budget	4
Graphs	3. “Wayze”: An Interactive Navigator	6

Sorting

Problem 1

The Scrambled Queue, From the Back. N people stood in a single line (front to back) waiting to board. A fire alarm scattered them. Your job is to reconstruct the original line. You may ask each person exactly two questions:

- “How tall are you?”
- “Among the people who were **behind** you in line, how many were **shorter** than you?”

Write $h(p)$ for p ’s height and $b(p)$ for p ’s answer to the second question. Assume all heights are distinct and all answers are truthful. Number the line’s slots $1, 2, \dots, N$ from the **front**.

The proposed algorithm (REBUILD). Process the people in *decreasing* order of height. To place the i -th tallest person p : scan the slots **right to left**, and put p in the $(b(p) + 1)$ -st **unoccupied** slot encountered.

(a) **Simulation and justification of the rule.**

- (i) Run REBUILD on the instance below. Show the state of the seven slots after *each* placement, not just the final line. State the processing order you used.

Person	$h(p)$	$b(p)$
Ama	71	3
Bo	64	0
Cyd	68	4
Dev	59	0
Eli	73	5
Fen	66	1
Gia	61	0

- (ii) Explain in two or three sentences *why* the tallest person’s slot is determined outright by $b(p)$ alone, and why that is no longer true for the second-tallest.
- (iii) A classmate proposes the mirror-image rule: process in *increasing* height and scan *left to right*. Does that rule reconstruct the same line from the same data $\{(h(p), b(p))\}$? Justify.

(b) **Complexity, and getting to $\Theta(N \log N)$.**

- (i) Give the worst-case running time of REBUILD as written, assuming the slots are a plain array with an **occupied** flag and the initial sort is any $\Theta(N \log N)$ comparison sort. Identify which step dominates and exhibit an input family that forces the worst case.
- (c) **Loop invariant.** Let $p_1, p_2, \dots, p_N$ be the people in decreasing order of height, and let L^* denote the true original line. Consider the loop of REBUILD that places $p_1, \dots, p_N$ in that order.
- (i) State a loop invariant strong enough to prove correctness. (Hint: it should say something about *which* slots are occupied at the top of iteration i , and it must say something about the people who have *not* yet been placed.)

Hashing

Problem 2

Read this first. The parts of this problem simulate *one* hash table over its lifetime. Every insertion is permanent, and each part inherits the table exactly as the previous part left it. An error in part (a) propagates. Verify each part before moving on. Lookups, load factors and probe counts are queries: they never modify the table.

The table. An open-addressed table with m slots, initially $m = 13$ (prime). Slots are $0, 1, \dots, m - 1$; each is **EMPTY** or holds a key.

The hash functions. Fix the prime $p = 1009$. For a table of size m ,

$$h_1(k) = ((41k + 17) \bmod p) \bmod m, \quad h_2(k) = 1 + (((23k + 5) \bmod p) \bmod (m - 1)).$$

Insertion uses **double hashing**: $H(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$ for $i = 0, 1, 2, \dots$. Note that both functions depend on the *current* m ; a key rehashed into a larger table gets fresh values. All arithmetic is exact integer arithmetic. **Probe convention.** Inspecting the base address $H(k, 0)$ counts as probe 1. A **collision** is any probe that inspects an *occupied* slot.

The rebuild rule (given; use it as stated). The table maintains a counter C , the total number of collisions since the last rebuild. Immediately after each insertion completes, the table rebuilds if *either*

$$(R1) \quad \alpha = \frac{n}{m} > \frac{1}{2}, \quad \text{or} \quad (R2) \quad C \geq 5.$$

A rebuild allocates a fresh table of size $m' =$ the smallest prime $\geq 2m$, sets $C \leftarrow 0$, and reinserts all keys using the same logic from a Cuckoo hashing rebuild. Reinsertion uses the same double-hashing rule with the new m . Collisions incurred *during* the rebuild count toward the new C . The rule is not re-checked until the rebuild finishes.

The dataset. $D = \langle 187, 247, 204, 867, 450, 858 \rangle$, then later $\langle 119, 275 \rangle$. Insert in the order listed.

(a) **Filling the first table.** Starting from an empty table with $m = 13$, insert the first six keys.

- (i) Give a trace table with one row per key and columns: k , $h_1(k)$, $h_2(k)$, the full sequence of slots inspected (probes), the landing slot, the collisions caused by this insertion, and the running C and α after it. Also fill out the hash table below:

slot	0	1	2	3	4	5	6	7	8	9	10	11	12
key													

- (ii) Which rebuild rule fired, and after which key? Was the *other* rule was also satisfied at time of rebuild?

(b) **The rebuild, and two more keys.**

- (i) State m' and justify it.
- (ii) State the reinsertion order explicitly as a list of six keys, each labelled with the old slot it came from. An error here invalidates the rest of the part.
- (iii) Perform the rebuild. Give a trace table (k , h_1 , h_2 , probe sequence, landing slot, probes) and report C at the end of the rebuild. Then fill in the diagram.

slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
key															

slot	15	16	17	18	19	20	21	22	23	24	25	26	27	28
key														

- (iv) Now insert 119 and then 275. Give their trace rows. Then answer: does the rebuild rule fire again? If so, state which clause, at what α . If not, what would m'' be if it had fired?

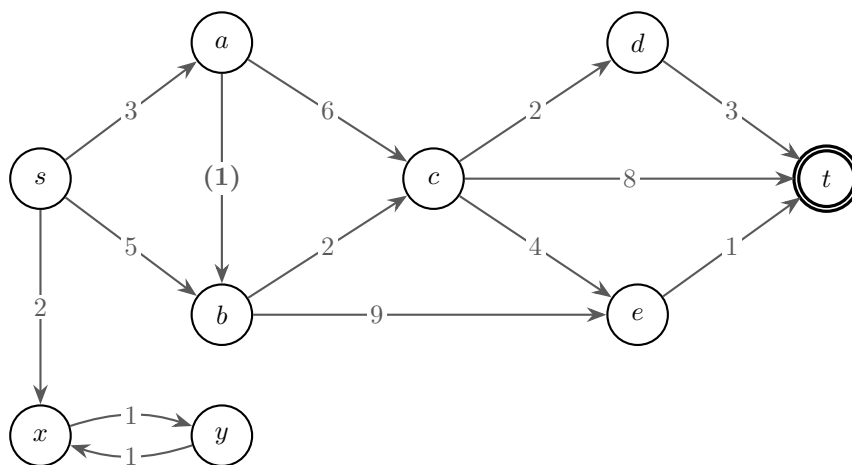
Graphs

Problem 3

“Wayze”: An Interactive Navigator. You are building a turn-by-turn navigation app. The road network is a directed graph $G = (V, E)$ with non-negative travel times $w(u, v)$. The user enters a source s and a destination t , and the app then enters an interactive loop. At each step the user is standing at some vertex u , and the app must:

- list all outgoing edges of u as the available moves;
- **recommend** every move that keeps the user on a shortest path to t ;
- flag any move that makes t unreachable (a dead end);
- after the user picks a move to x , report either “*Good choice*” or “*If you say so*”, together with the minimum remaining distance from x to t .

Hard requirement. Dijkstra may **not** be re-run inside the interactive loop. The work done at each step must be $\Theta(\deg^+(u))$, where $\deg^+(u)$ is the out-degree of the user’s current vertex.



Destination t is double-circled, as you would learn if you took CSE 105. It stands for a ‘final state’.

(a) The algorithm.

- Describe the precomputation. Say exactly which graph you run Dijkstra on and from which vertex, and what quantity the resulting labels hold.
- Give the per-step rule as pseudocode: given the current vertex u and the precomputed labels, how do you classify each outgoing edge as RECOMMENDED, SUBOPTIMAL (with a penalty), or DEAD END?
- Carry out the precomputation on the graph above with t as the destination. Report the label of every vertex, including the ones whose label is ∞ .
- Simulate this session, giving the app’s full response at each step (the option list with each option’s classification, the recommendation set, the verdict, and the remaining distance): the user starts at s and chooses, in order, $s \rightarrow b$, then $b \rightarrow c$, then $c \rightarrow e$, then $e \rightarrow t$. Report the user’s total travel time and compare it to $\text{dist}(s, t)$.
- Separately: at s , what should the app say about the option $s \rightarrow x$? Justify from your labels.

(b) Complexity.

- (i) Give the precomputation time and space with a binary-heap Dijkstra, in terms of $|V|$ and $|E|$. Include the cost of building whatever auxiliary structure you need.
- (ii) Prove that the work at a single interactive step at vertex u is $\Theta(\deg^+(u))$, thereby meeting the hard requirement. Be explicit about which lookups are $O(1)$ and why.